

超越 Pod: 为大规模模型即服务 (MaaS) 重构 Kubernetes 编排范式

王夕宁 | 阿里云 | 大模型服务平台百炼 Tech Lead



Content 目录

01 MaaS 时代的挑战

02 范式转变：从 K8s 原生范式到三层协同

03 数据平面的动态调度与运行时管理

04 总结与展望



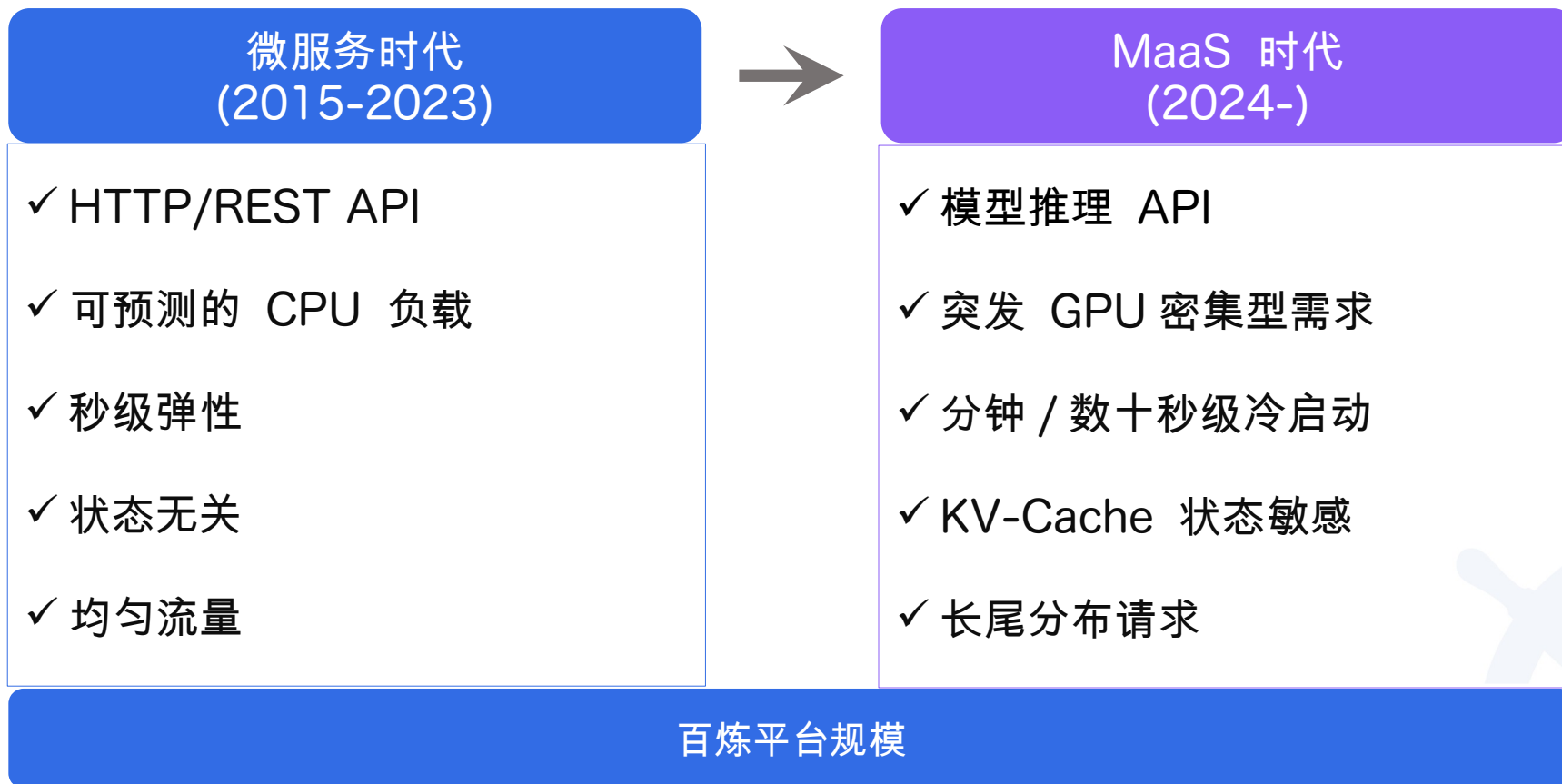
Part 01

MaaS 时代的挑战





MaaS 时代的到来



★ 每日数万亿
tokens

★ 数千种异构模型

★ 跨地域 GPU 集群



✂ 核心矛盾：Kubernetes vs MaaS

K8s 设计假设

- ✗ Pod 是调度单元
- ✗ 请求是均匀的
- ✗ 服务是无状态的
- ✗ HPA 基于 CPU/ 内存
- ✗ 冷启动秒级可接受



MaaS 现实

- ✓ 模型是调度单元
- ✓ 请求长度变化 1000x
- ✓ KV-Cache 是核心状态
- ✓ 需要基于 GPU 显存
- ✓ 冷启动数十秒 = SLO 违约

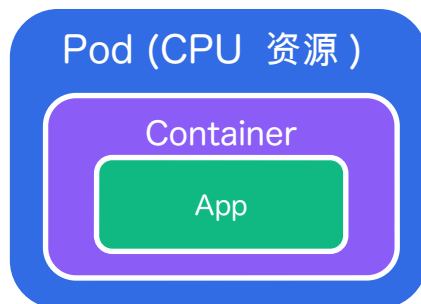
未优化结果：资源利用率 <40% | KV-Cache 复用率 <30% | 模型切换延迟 30-60 秒



🔍 问题根源：抽象粒度过粗

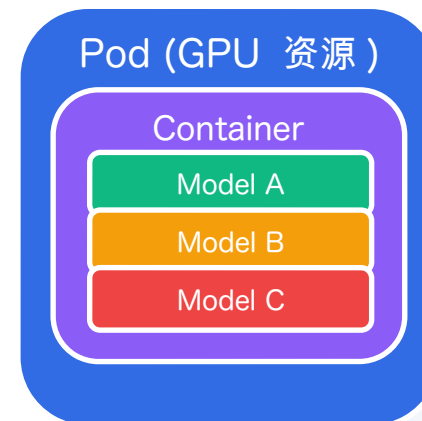
传统微服务架构

- Pod = 调度单元 = 服务单元
- Container 与 App 1:1 映射



MaaS 架构需求

- Pod = GPU 资源调度单元
- 多个模型进程动态切换
- K8s 无法感知 Pod 内部状态



Part 02

范式转变：从 K8s 原生范式到三层协同

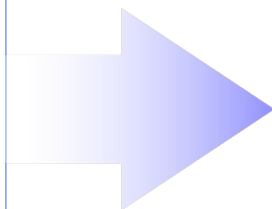


💡 解决方案：范式转变

从「Kubernetes 原生调度范式」到「三层协同调度范式」

传统架构

- 控制平面集中调度
- Pod = 调度单元
- 请求无感知
- 进程不可见
- 粗粒度（秒级）



MaaS 新架构

- 三层协同：控制平面 + 数据平面 + 运行时平面
- 模型 = 调度单元
- 请求级智能调度
- 进程级管理
- 细粒度（毫秒级）



架构蓝图：三层协同设计

架构分层

控制平面

策略配置、CRD 管理、全局协调

数据平面

请求感知调度、KV-Cache 路由、会话亲和性

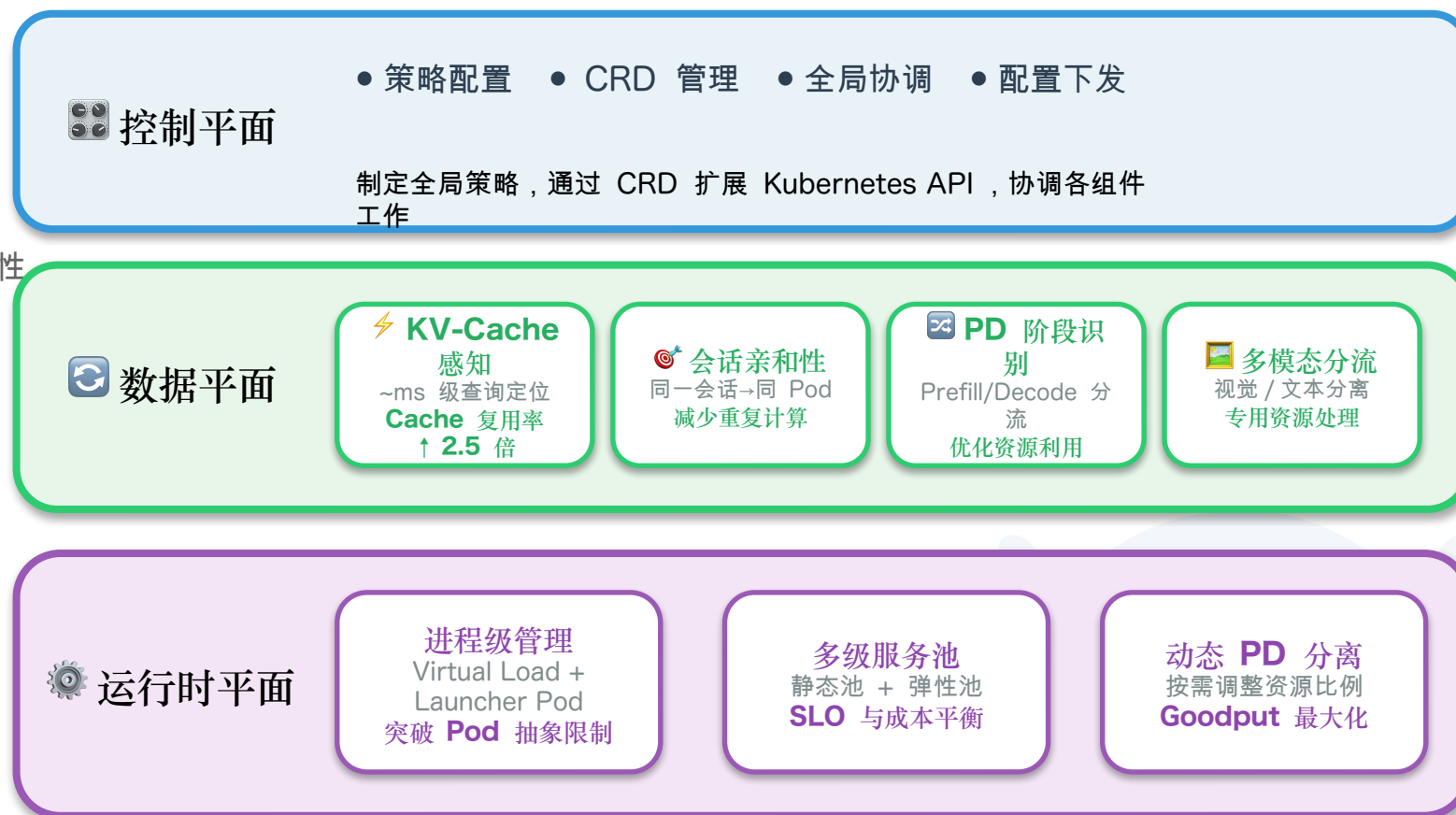
运行时平面

进程级管理、多级服务池、动态 PD 分离

核心价值

- ✓ 请求级调度粒度 (vs Pod 级)
- ✓ KV-Cache 复用率提升 * 倍
- ✓ 模型切换延迟降低 **%
- ✓ 资源利用率提升 * 倍

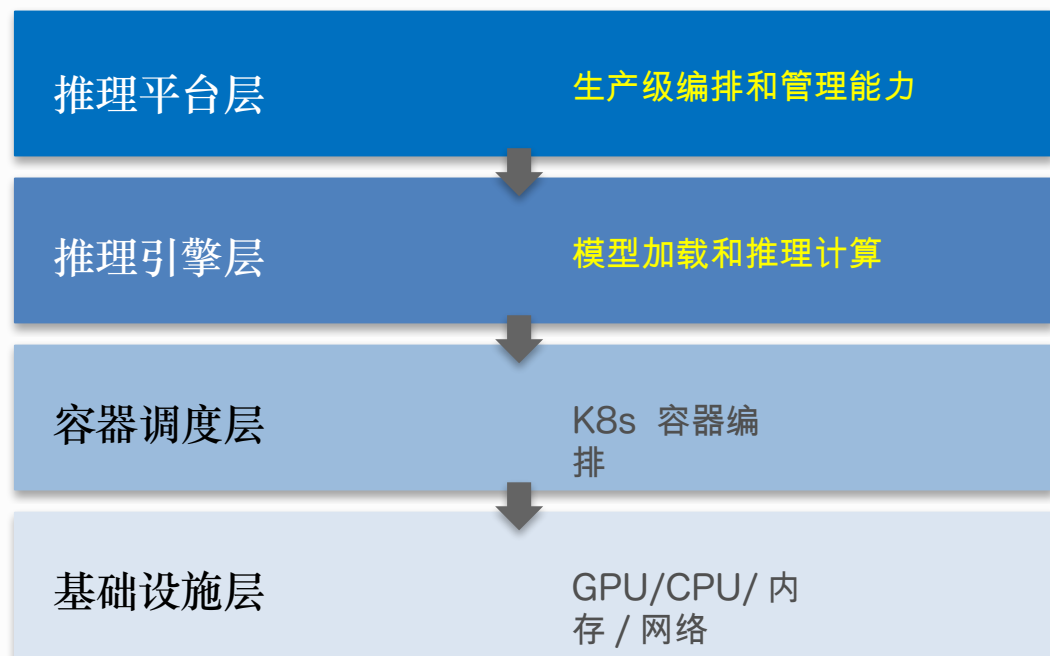
注意：性能数据因环境不同会有差异



🎯推理平台定位：位于推理引擎之上的模型服务基础设施层 VLLM

- 推理平台：提供生产级编排和管理能力 | 推理引擎：专注模型加载和推理计算

平台架构分层



推理平台核心能力

- 多模型管理
- 多引擎支持
- 智能路由与流量管理
- 架构解耦与 PD 动态分离
- 分布式编排
- 全局 KV Cache 管理
- 资源调度与编排
- 弹性伸缩
- 监控与可观测性
- 多租户与安全隔离
- 高可用保障
- 成本优化

推理平台向上支撑业务应用，向下管理推理引擎和基础设施



大模型部署抽象定义 ModelDeployment

LLM



$\text{ModelDeployment} \supseteq \{ \text{Runtime}, \text{Model}, \text{Resources}, \text{Component}, \text{Configs} \}$

Runtime

- 推理引擎核心
Engine
- 服务模式
ServiceMode
- 调试配置
Debug
- 运行时参数
Params

Model

- Selection
模型选择
- MountedModels[
挂载模型列表
]

Resources

- 计算资源
Compute
- 多节点配置
MultiNode
- 实例管理
Instances

Component

- 调度器
Scheduler
- KV 缓存管理
KVCache
- Baseline
基线组件
- 健康检查
Health

Configs

- Basic
基础配置
- 环境变量
Env
- 扩展配置
Extensions
- 依赖管理
Deps

**** 把模型作为调度单元，而不是 Pod 本身 ****

Part 03

数据平面的动态调度与运行时管理





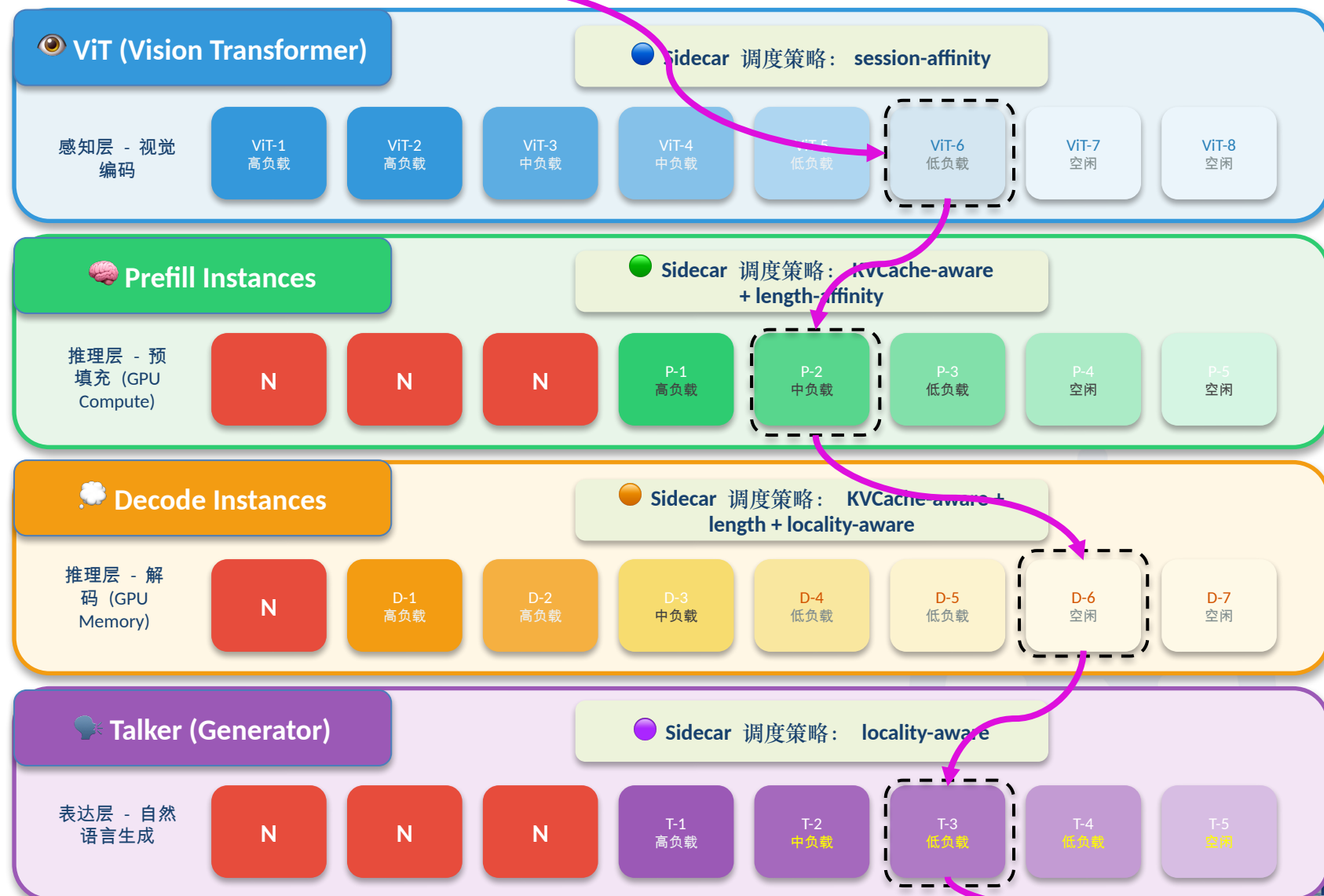
多模态推理服务架构：基于 Mesh 的多阶段智能请求调度

LLM

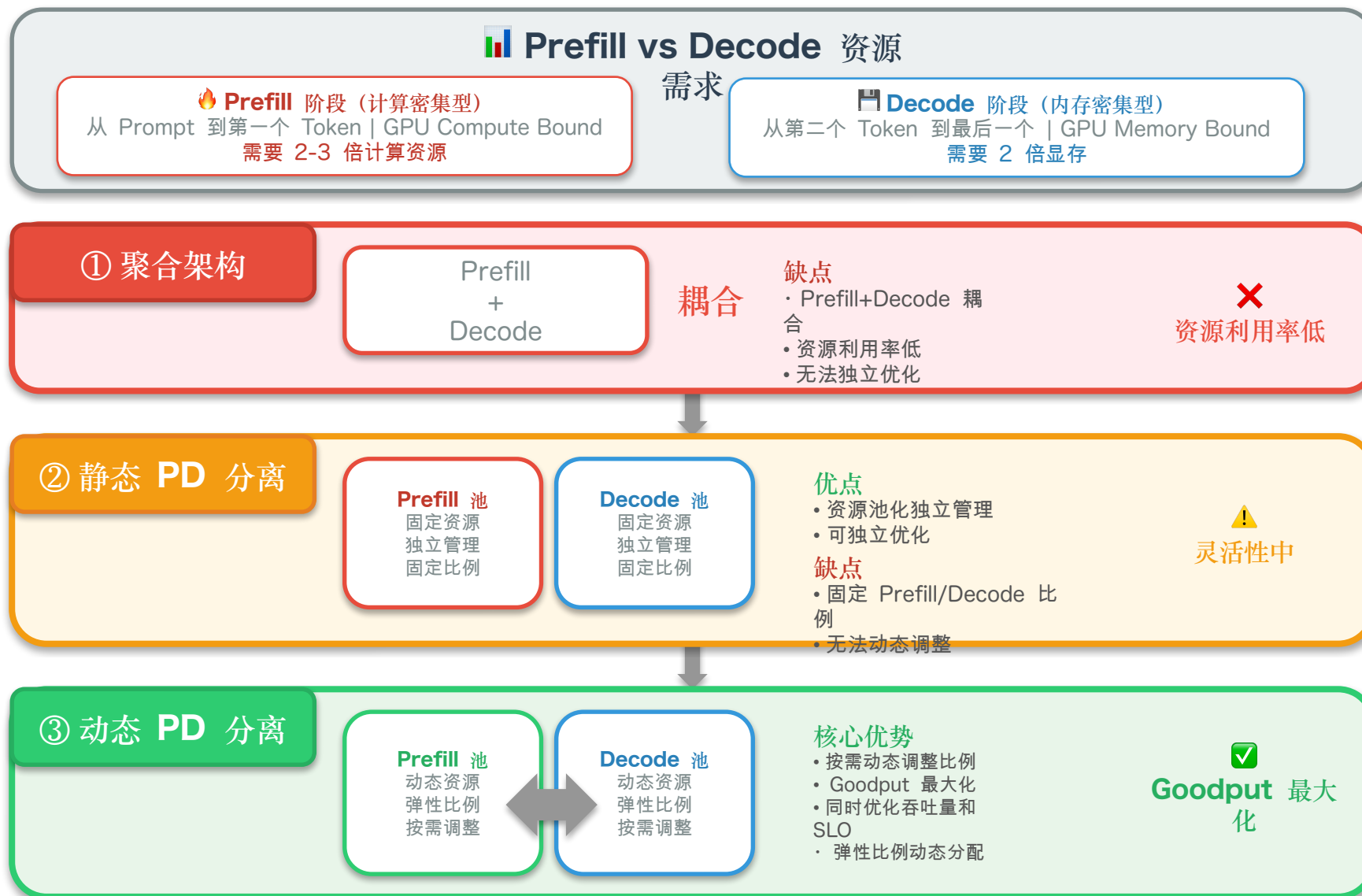


感知 → 推理 → 表达：每个阶段通过 Sidecar 实现差异化调度策略

- KV-Cache 感知 (~ms)
- 会话亲和性路由 (同一会话 → 同一 Pod)
- 预填充 / 解码阶段识别 (分流优化)
- 多模态任务分流 (视觉 / 文本分离)



PD 分离架构演进：从静态到动态





进程级动态运行时管理

✗ 传统 Pod 抽象限制

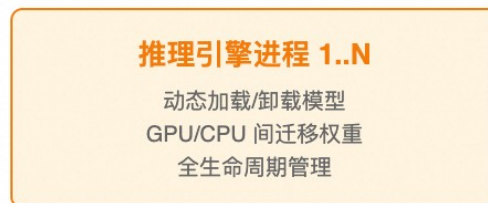
- Pod 捆绑镜像/资源/配置
- 部署方案僵化
- 服务与资源绑定



LLM 推理场景
需要动态灵活

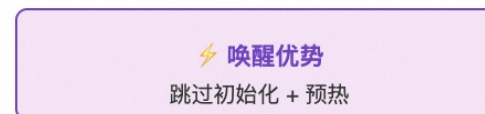
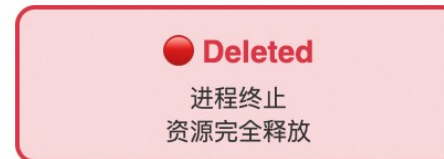
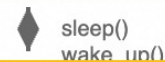
资源利用率 < 40%

✓ 进程级管理架构



- ✓ 资源与服务解耦
- ✓ 动态灵活调度

进程状态动态切换



效果：模型切换延迟 30-60 秒 → 3-5 秒（降低 90%+） | 资源利用率提升 3.5 倍



多级服务池架构

在 SLO 保障与资源利用率之间找到最佳平衡点

LLM



真实工作负载特征

10% 模型服务 90% 请求 (长尾分布) | 几分钟内负载激增 2-3 倍 (突发流量) | 不同请求的 TTFT SLO 约束不同

静态推理服务池 (高优先级)

服务对象

- 高流量热点模型
- 高优先级租户
- 严格 SLO 约束

资源保障

- 非抢占式固定资源
- 维持运行无释放

SLO 保障

- 彻底规避冷启动
- TTFT < 500ms
- 可用性 > 99.9%

资源保障

固定分配
不释放
无冷启动

动态弹性推理服务池 (低优先级)

服务对象

- 偶发突发流量
- 长尾模型
- 低优先级请求

资源共享

- 抢占式分配
- 低负载时释放

成本优化

- 进程级快速唤醒
- 应对动态负载
- 资源利用率最大化

成本优化

抢占式
可释放
快速唤醒

全局调度协调器

- 池间资源借用
- 溢出处理
- 动态调整

静态池空闲资源可临时借给弹性池
弹性池负载溢出时自动扩展到静态池
基于优先级和流量热度加权, 线性规划最小化启动开销



从零扩容：突破 HPA 限制

HPA 局限性

- 副本数为 0 时，Pod 指标不可获得，被 Kubernetes HPA 忽略
- 较难触发从零扩容
- 服务首次流量无法响应

V
S

解决方案

- 管理初始副本数为 0 的 Virtual Load Deployment
- 推理网关流量监测组件暴露实时请求流量
- 基于请求排队长度触发突发扩容
- 依据：基于队列深度、待处理 Token 量、GPU 压力等指标；
- Prefill/Decode Pod 独立智能伸缩



Part 04

总结与展望





总结与展望

🎯 欢迎联系交流，共同探索最佳实践

1

Pod 抽象局限

Kubernetes 的 Pod 抽象在 MaaS 场景下存在根本性局限

✗ 粒度过粗

2

资源服务解耦

虚拟负载 + 进程级管理，计算资源与服务配置解耦

✓ 解耦设计

3

动态唤醒机制

模型切换延迟降低 90%，TTFT 降低 61-88%

↓ 90% ↓ 61-88%

4

多级服务池 + PD 分离

平衡 SLO 保障与资源利用率，资源利用率提升 2.5-3.5 倍

↑ 2.5-3.5 倍

Thanks.

